Enabling People
Education for Future Generations

Curso de Inteligencia Artificial

Capítulo 3. Quiz

Para estudiantes

©2023 SAMSUNG. All rights reserved.
Samsung Electronics Corporate Citizenship Office holds the copyright of this document.
This document is a literary property protected by copyright law so reprint and reproduction without permission are prohibited.
To use this document other than the curriculum of Samsung Innovation Campus, you must receive written consent from copyright holder.

Samsung Innovation Campus

Curso de Inteligencia Artificial

Capítulo 3. Quiz

Alumno: Oscar Gomez Vasquez
Correo: oscargv.1324@gmail.com

In [1]:
```
from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

# Carpeta data/ en la raíz del repo: subimos de carpeta en carpeta hasta dar con pyproject.toml
raiz = next((p for p in [Path.cwd(), *Path.cwd().parents] if (p / "pyproject.toml").exists()), Path.cwd())
DATA_DIR = raiz / "data"
```

---

1. Responde las siguientes preguntas usando funciones de la librería pandas.

In [2]:
```
df_estudiantes = pd.read_csv(DATA_DIR / "data_studentlist_en.csv", header="infer")
df_estudiantes.head(5)
```

Out[2]:

	Name	Gender	Age	Grade	Absence	Bloodtype	Height	Weight
0	Jared Diamond	M	23	3	Y	O	165.3	68.2
1	Sarah O'Donnel	F	22	2	N	AB	170.1	53.0
2	Brian Martin	M	24	4	N	B	175.0	80.1
3	David Hassel	M	23	3	N	AB	182.1	85.7
4	Clara Rodriquez	F	20	1	Y	A	168.0	49.5

In [3]:
```
hombres = df_estudiantes[df_estudiantes["Gender"] == "M"]
mujeres = df_estudiantes[df_estudiantes["Gender"] == "F"]
```

1) Altura promedio de los estudiantes varones:

In [4]:
```
hombres["Height"].mean()
```

Out[4]:
```
np.float64(172.41000000000003)
```

2) Altura promedio de las estudiantes:

In [5]:
```
mujeres["Height"].mean()
```

Out[5]: np.float64(166.64285714285714)

3) Peso promedio de los estudiantes varones:

In [6]:
```
hombres["Weight"].mean()
```

Out[6]: np.float64(68.50000000000001)

4) Peso promedio de las estudiantes:

In [7]:
```
mujeres["Weight"].mean()
```

Out[7]: np.float64(50.44285714285714)

5) La mayor estatura entre los estudiantes varones:

In [8]:
```
hombres["Height"].max()
```

Out[8]: np.float64(182.1)

6) La menor estatura entre las estudiantes:

In [9]:
```
mujeres["Height"].min()
```

Out[9]: np.float64(155.2)

7) El peso más ligero entre los estudiantes varones de estatura superior a 1.75 m:

(La estatura está en cm: 1.75 m = 175.)

In [10]:
```
hombres[hombres["Height"] > 175]["Weight"].min()
```

Out[10]: np.float64(61.3)

8) El mayor peso entre las estudiantes de estatura inferior a 1.60 m:

In [11]:
```
mujeres[mujeres["Height"] < 160]["Weight"].max()
```

Out[11]: np.float64(45.3)

9) La nota promedio de los alumnos (hombres y mujeres) sin ausencias ('N'):

In [12]:
```
df_estudiantes[df_estudiantes["Absence"] == "N"]["Grade"].mean()
```

Out[12]:
```
np.float64(2.076923076923077)
```

10) La nota promedio de los alumnos (hombres y mujeres) con absentismo ('Y'):

In [13]:
```
df_estudiantes[df_estudiantes["Absence"] == "Y"]["Grade"].mean()
```

Out[13]:
```
np.float64(3.0)
```

11) Altura promedio de los alumnos (hombres y mujeres) con grupo sanguíneo 'A' o 'AB':

In [14]:
```
df_estudiantes[df_estudiantes["Bloodtype"].isin(["A", "AB"])]["Height"].mean()
```

Out[14]:
```
np.float64(172.64285714285714)
```

12) Altura promedio de los alumnos varones con grupo sanguíneo 'A' o 'AB':

In [15]:
```
hombres[hombres["Bloodtype"].isin(["A", "AB"])]["Height"].mean()
```

Out[15]:
```
np.float64(173.375)
```

13) Edad promedio de los alumnos (hombres y mujeres) con absentismo ('Y') cuya nota es mayor o igual a 3:

In [16]:
```
df_estudiantes[(df_estudiantes["Absence"] == "Y") & (df_estudiantes["Grade"] >= 3)]
["Age"].mean()
```

Out[16]:
```
np.float64(23.666666666666668)
```

---

2. Responde las siguientes preguntas usando funciones de las librerías pandas y numpy.

In [17]:
```
# thousands="," convierte textos como " 10,078,850 " en números
df_censo = pd.read_csv(DATA_DIR / "census.csv", header="infer", thousands=",")
print(df_censo.shape)
df_censo.head(10)
```

```
(281, 6)
```

Out[17]:

	Province Code	City Code	Population	Households	Person's Household	Gender Ratio
0	109	1001	10078850	4197478	2.40	0.97
1	109	1188	155695	72882	2.14	0.98
2	109	1189	126817	59614	2.13	1.00
3	109	1158	235186	108138	2.17	0.94
4	109	1105	298145	126915	2.35	0.99
5	109	1032	362197	158769	2.28	0.97
6	109	1065	362604	160110	2.26	1.01
7	109	1190	417976	177077	2.36	1.00
8	109	1106	464176	192670	2.41	0.96
9	109	1006	333523	141325	2.36	0.97

1) Población total con ProvinceCode igual a 115 o 116:

In [18]:
```
df_censo[df_censo["Province Code"].isin([115, 116])]["Population"].sum()
```

Out[18]:
```
np.int64(5970327)
```

2) Población promedio de las ciudades donde hay más hombres (GenderRatio > 1):

In [19]:
```
df_censo[df_censo["Gender Ratio"] > 1]["Population"].mean()
```

Out[19]:
```
np.float64(419221.2076923077)
```

3) Lugares con más hombres (GenderRatio > 1) y menos de 2 personas por hogar:

In [20]:
```
df_censo[(df_censo["Gender Ratio"] > 1) & (df_censo["Person's Household"] < 2)]
```

Out[20]:

	Province Code	City Code	Population	Households	Person's Household	Gender Ratio
62	112	1155	20825	11172	1.86	1.29
149	101	1240	26790	13433	1.99	1.27
152	101	1017	30066	15485	1.94	1.15
167	117	1033	38027	19145	1.99	1.03
227	113	1120	43566	22248	1.96	1.11
253	104	1163	10304	5365	1.92	1.15

4) Ordena el DataFrame en orden ascendente de 'Households' y muestra los 10 primeros.

In [21]:
```
df_censo.sort_values(by="Households", ascending=True).head(10)
```

Out[21]:

	Province Code	City Code	Population	Households	Person's Household	Gender Ratio
253	104	1163	10304	5365	1.92	1.15
244	104	1147	17984	8769	2.05	0.97
200	114	1179	23308	10877	2.14	0.98
150	101	1130	24283	11162	2.18	1.17
62	112	1155	20825	11172	1.86	1.29
199	114	1073	25181	11723	2.15	0.98
241	104	1039	24096	12091	1.99	1.00
213	113	1034	27115	12465	2.18	0.95
198	114	1193	26325	12531	2.10	1.00
243	104	1208	26353	13233	1.99	0.98

---

3. Responde las siguientes preguntas.

In [22]:
```
df_ventas = pd.read_csv(DATA_DIR / "sales.csv", header="infer")
print(df_ventas.shape)
df_ventas.head(5)
```

```
(43, 5)
```

Out[22]:

	Date	Region	Item	Unit Price	Units
0	04-Jul-14	East	Pen Set	4.99	62
1	12-Jul-14	East	Binder	1.99	29
2	21-Jul-14	Central	Pen Set	12.49	55
3	29-Jul-14	East	Binder	19.99	81
4	07-Aug-14	Central	Pen Set	23.95	42

1) Añade una nueva variable Amount = UnitPrice x Units y muestra las 5 primeras observaciones.

In [23]:
```
df_ventas["Amount"] = df_ventas["Unit Price"] * df_ventas["Units"]
df_ventas.head(5)
```

Out[23]:

	Date	Region	Item	Unit Price	Units	Amount
0	04-Jul-14	East	Pen Set	4.99	62	309.38
1	12-Jul-14	East	Binder	1.99	29	57.71
2	21-Jul-14	Central	Pen Set	12.49	55	686.95
3	29-Jul-14	East	Binder	19.99	81	1619.19
4	07-Aug-14	Central	Pen Set	23.95	42	1005.90

2) Calcula el precio unitario promedio de cada región. Utiliza el método groupby().

In [24]:
```
grupo_region = df_ventas.groupby("Region")
grupo_region["Unit Price"].mean()
```

Out[24]:
```
Region
Central    18.018750
East        9.143846
West       53.658333
Name: Unit Price, dtype: float64
```

3) Calcula el precio unitario promedio de cada región. Utiliza el método pivot_table().

In [25]:
```
df_ventas.pivot_table(index="Region", values="Unit Price", aggfunc="mean")
```

Out[25]:

Region	Unit Price
Central	18.018750
East	9.143846
West	53.658333

4) Calcula el precio unitario promedio para cada región en una línea de código. Utiliza el método groupby().

In [26]:
```
df_ventas.groupby("Region")["Unit Price"].mean()
```

Out[26]:
```
Region
Central    18.018750
East        9.143846
West       53.658333
Name: Unit Price, dtype: float64
```

5) Calcula el precio unitario promedio y las unidades para cada región en una línea de código. Utiliza el método pivot_table().

In [27]:
```
df_ventas.pivot_table(index="Region", values=["Unit Price", "Units"], aggfunc=
{"Unit Price": "mean", "Units": "sum"})
```

Out[27]:

Region	Unit Price	Units
Central	18.018750	1199
East	9.143846	691
West	53.658333	231

6) Calcula el total de unidades para cada región y tipo de artículo en una línea de código. Utiliza el método pivot_table() y rellena los valores faltantes con 0.

In [28]:
```
df_ventas.pivot_table(index="Region", columns="Item", values="Units",
aggfunc="sum", fill_value=0)
```

Out[28]:

Item	Binder	Desk	Pen	Pen Set	Pencil
Region					
Central	424	7	27	243	498
East	234	0	175	152	130
West	64	3	76	0	88

7) Calcula el importe total de ventas para cada región y el tipo de artículo en una línea de código. Utiliza el método pivot_table() y rellena los valores faltantes con 0.

In [29]:
```
df_ventas.pivot_table(index="Region", columns="Item", values="Amount",
aggfunc="sum", fill_value=0)
```

Out[29]:

Item	Binder	Desk	Pen	Pen Set	Pencil
Region					
Central	5762.63	875.0	539.73	2421.39	1540.32
East	2535.66	0.0	1354.25	1748.48	363.70
West	1279.36	825.0	151.24	0.00	231.12

---

4. Responde las siguientes preguntas.

In [30]:
```
df_cafeterias = pd.read_csv(DATA_DIR / "coffeeshop.csv", header="infer", na_values=
[" "])
print(df_cafeterias.shape)
df_cafeterias.head(5)
```

```
(43182, 3)
```

Out[30]:

	Year of Start	Current State	Size of Site
0	2008.0	In	20.80
1	2010.0	In	212.72
2	2013.0	In	20.04
3	2012.0	In	64.17
4	2002.0	Out	11.40

1) Genera una tabla de frecuencias para la variable 'yearOfStart' y visualízala por año.

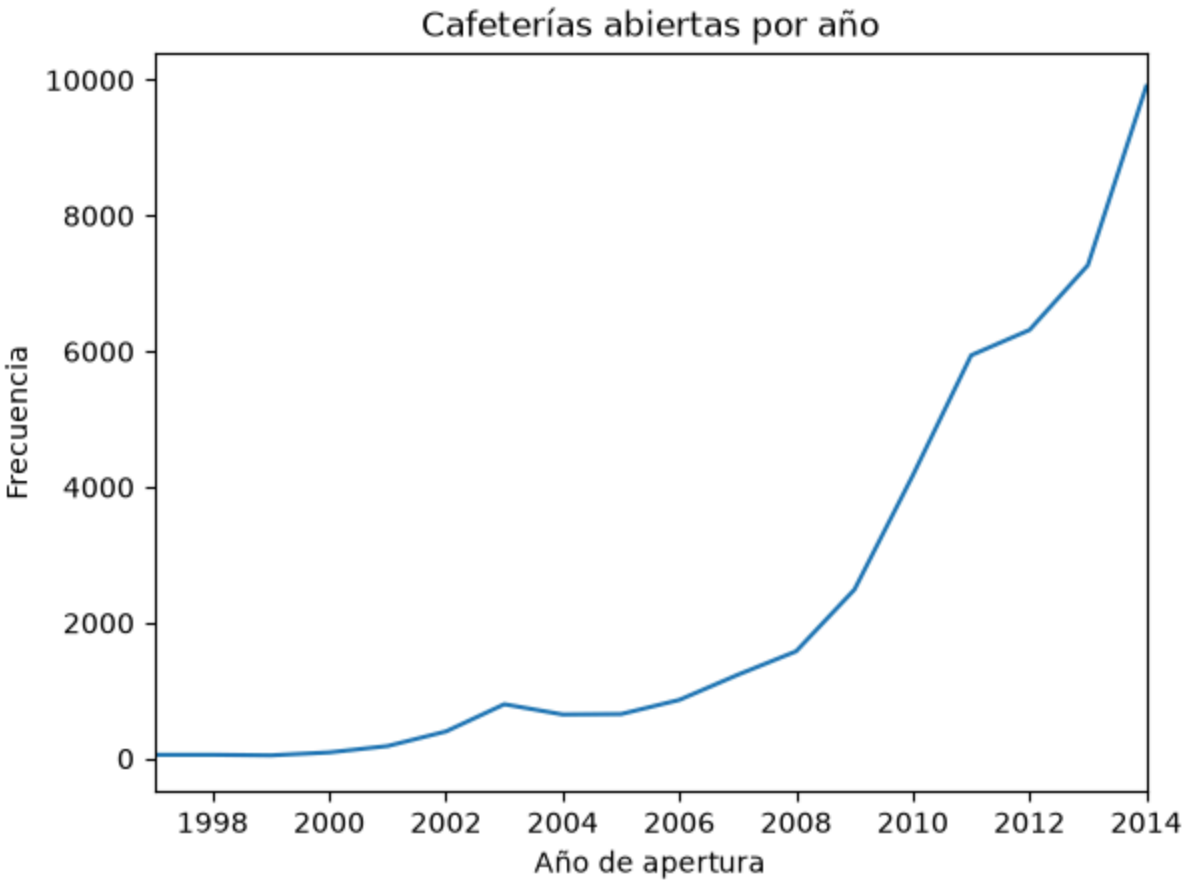
- **Ordena por año.**
- **Dibuja una gráfica de línea de 1997 a 2014. <= Pista: plt.xlim()**

In [31]:
```
frecuencia_anual = df_cafeterias["Year of Start"].value_counts().sort_index()
frecuencia_anual
```

Out[31]:
```
Year of Start
1964.0       2
1966.0       2
1967.0       3
1968.0       1
1969.0       2
1970.0       4
1971.0       6
1972.0       3
1974.0       1
1975.0       2
1976.0       5
1979.0       4
1980.0       9
1981.0       8
1982.0      12
1983.0       9
1984.0      11
1985.0      18
1986.0      21
1987.0      21
1988.0      26
1989.0      23
1990.0      25
1991.0      28
1992.0      37
1993.0      50
1994.0      48
1995.0      48
1996.0      41
1997.0      54
1998.0      54
1999.0      46
2000.0      89
2001.0     183
2002.0     398
2003.0     799
2004.0     648
2005.0     654
2006.0     863
2007.0    1233
2008.0    1579
2009.0    2489
2010.0    4172
2011.0    5942
2012.0    6315
2013.0    7270
2014.0    9905
Name: count, dtype: int64
```

In [32]:
```
frecuencia_anual.plot(kind="line")
plt.xlim(1997, 2014)
plt.title("Cafeterías abiertas por año")
```

```
plt.xlabel("Año de apertura")
plt.ylabel("Frecuencia")
plt.show()
```

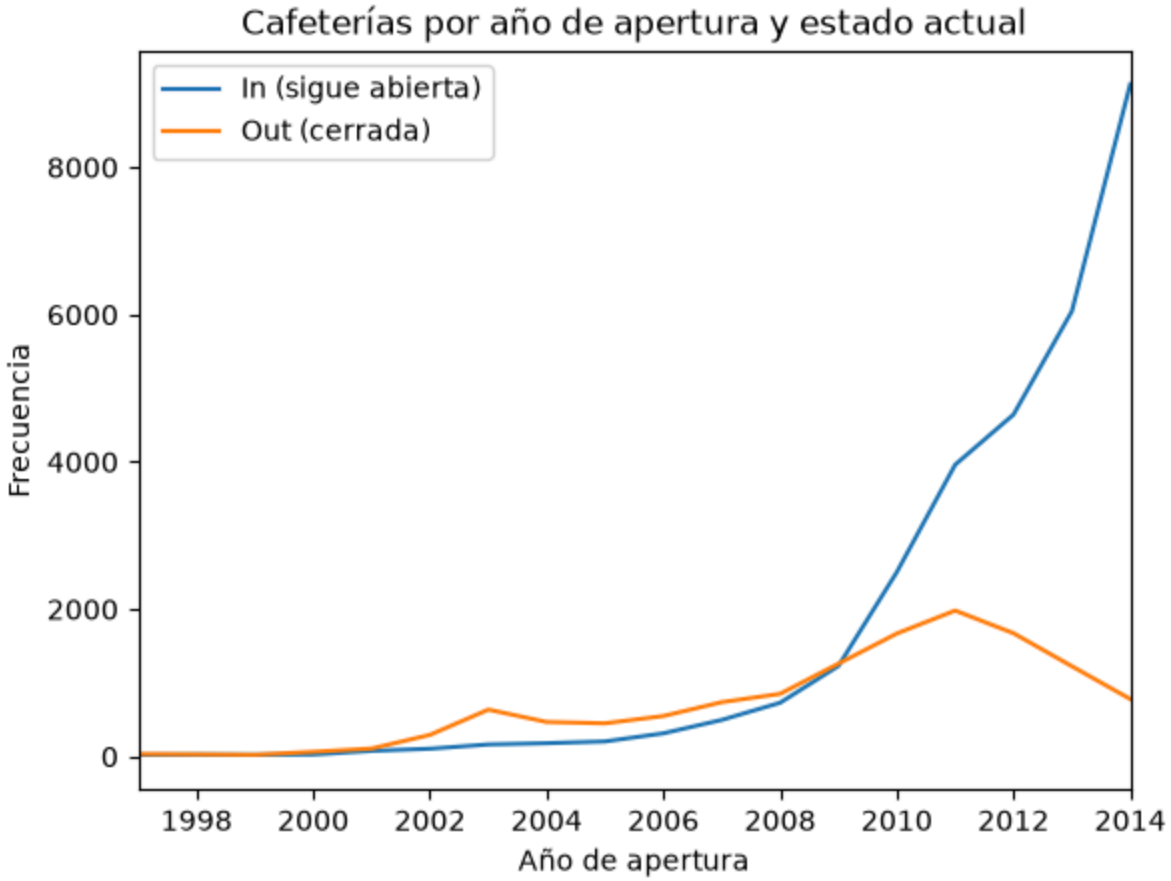


2) Divide los datos por el estado actual del negocio ('In' o 'Out'). Después, visualiza la tendencia anual de las frecuencias de la variable 'yearOfStart'.

- **Ordena por año.**
- **Dibuja dos gráficas de líneas superpuestas de 1997 a 2014.**
- **Usa el objeto 'figure'.**

In [33]:
```
abiertas = df_cafeterias[df_cafeterias["Current State"] == "In"]["Year of
Start"].value_counts().sort_index()
cerradas = df_cafeterias[df_cafeterias["Current State"] == "Out"]["Year of
Start"].value_counts().sort_index()

fig = plt.figure()
ax = fig.add_subplot(1, 1, 1)
ax.plot(abiertas, label="In (sigue abierta)")
ax.plot(cerradas, label="Out (cerrada)")
ax.set_xlim(1997, 2014)
ax.set_title("Cafeterías por año de apertura y estado actual")
ax.set_xlabel("Año de apertura")
ax.set_ylabel("Frecuencia")
ax.legend()
plt.show()
```



3) A partir de los resultados de 1) y 2), ¿cómo describirías la tendencia en general?

Respuesta: El número de cafeterías que abren cada año crece de forma constante desde 1997 y se acelera mucho a partir de 2008-2009: en 2014 abrieron más de 9,900. Las cafeterías antiguas se cerraron en su mayoría: en casi todos los años hasta 2009 hay más negocios cerrados ('Out') que abiertos ('In'). En cambio, desde 2010 la mayoría de las cafeterías nuevas siguen abiertas, y las cerradas bajan a partir de 2011. Esto tiene sentido porque los negocios más recientes han tenido menos tiempo para cerrar. En general, el mercado de las cafeterías está en plena expansión.

---

5. ¿Cuál es la sintaxis correcta para crear un arreglo de NumPy?

a) `np.createArray([1, 2, 3, 4, 5])` b) `np.array([1, 2, 3, 4, 5])` c) `np.object([1, 2, 3, 4, 5])`

In [34]:
```
np.array([1, 2, 3, 4, 5])
```

Out[34]:
```
array([1, 2, 3, 4, 5])
```

Respuesta: b) `np.array([1, 2, 3, 4, 5])`

6. ¿Cuál de las siguientes opciones es un arreglo de dos dimensiones?

a) `42` b) `([1, 2, 3], [4, 5, 6])` c) `[1, 2, 3, 4, 5]`

In [35]:
```
np.array(([1, 2, 3], [4, 5, 6])).ndim
```

Out[35]: 2

Respuesta: b) `([1, 2, 3], [4, 5, 6])`

7. ¿Cuál es la sintaxis correcta para comprobar el número de dimensiones de un arreglo?

a) `np.dim()` b) `np.ndim()` c) `np.ndim` d) `np.dim`

In [36]:
```
arr = np.array([[1, 2, 3], [4, 5, 6]])
arr.ndim  # ndim es un atributo: se escribe sin paréntesis
```

Out[36]: 2

Respuesta: c) `np.ndim`

8. ¿Cuál es la sintaxis correcta para imprimir el primer elemento de un arreglo?

a) `print(myArr[0])` b) `print(myArr,1)` c) `print(myArr[1])`

In [37]:
```
myArr = np.array([10, 20, 30])
print(myArr[0])  # los índices empiezan en 0
```

```
10
```

Respuesta: a) `print(myArr[0])`

9. ¿Cuál es la sintaxis correcta para imprimir el número 8 de la siguiente matriz?

```
arr = np.array([[1,2,3,4,5], [6,7,8,9,10]])
```

a) `print(arr[3, 0])` b) `print(arr[7, 2])` c) `print(arr[1, 2])`

In [38]:
```
arr = np.array([[1, 2, 3, 4, 5], [6, 7, 8, 9, 10]])
print(arr[1, 2])  # fila 1, columna 2
```

```
8
```

Respuesta: c) `print(arr[1, 2])`

10. ¿Cuál es la sintaxis correcta para imprimir los números [3, 4, 5] de la siguiente matriz?

```
arr = np.array([1,2,3,4,5,6,7])
```

a) `print(arr[2:4])` b) `print(arr[2:5])` c) `print(arr[2:6])` d) `print(arr[3:6])`

In [39]:
```
arr = np.array([1, 2, 3, 4, 5, 6, 7])
print(arr[2:5])  # el final del slice no se incluye
```

```
[3 4 5]
```

Respuesta: b) `print(arr[2:5])`

11. ¿Cuál es el método correcto para unir dos o más arreglos?

a) `array_join()` b) `concatenate()` c) `join()`

In [40]:
```
np.concatenate((np.array([1, 2]), np.array([3, 4])))
```

Out[40]:
```
array([1, 2, 3, 4])
```

Respuesta: b) `concatenate()`

12. ¿Cuál es la sintaxis correcta para devolver el índice de todos los elementos que tienen el valor 4 de la siguiente matriz?

```
arr = np.array([1,4,3,4,5,4,4])
```

a) `arr.search(4)` b) `arr.where()` c) `np.where(arr==4)`

In [41]:
```
arr = np.array([1, 4, 3, 4, 5, 4, 4])
np.where(arr == 4)
```

Out[41]:
```
(array([1, 3, 5, 6]),)
```

Respuesta: c) `np.where(arr==4)`

13. Cuando se utiliza el módulo random de NumPy, ¿cómo se puede devolver un número aleatorio entre 0 y 100?

a) `random.rand(100)` b) `random.rand()` c) `random.randint(100)`

In [42]:
```
np.random.seed(42)
np.random.randint(100)  # entero entre 0 y 99
```

Out[42]:
```
51
```

Respuesta: c) `random.randint(100)`

14. Cuando se utiliza el módulo random de NumPy, ¿cómo se puede devolver una Distribución Normal de Datos con 1000 números, concentrados alrededor del número 50 y con una desviación estándar de 0.2?

a) `random.normal(size=1000, loc=50, scale=0.2)` b) `random.normal(size=1000, mean=50, deviation=0.2)` c) `random.normal(size=1000, normal=50, s=0.1)`

In [43]:
```
np.random.seed(42)
x = np.random.normal(size=1000, loc=50, scale=0.2)  # loc = media, scale = desviación estándar
x.mean(), x.std()
```

Out[43]:
```
(np.float64(50.00386641116446), np.float64(0.1957452415494709))
```

Respuesta: a) `random.normal(size=1000, loc=50, scale=0.2)`

15. ¿Cuál es la sintaxis correcta para sumar matemáticamente los números de arr1 a los números de arr2?

a) `np.add(arr1, arr2)` b) `np.append(arr1, arr2)` c) `sum(arr1, arr2)`

In [44]:
```
arr1 = np.array([1, 2, 3])
arr2 = np.array([10, 20, 30])
np.add(arr1, arr2)
```

Out[44]:
```
array([11, 22, 33])
```

Respuesta: a) `np.add(arr1, arr2)`

16. ¿Cuál es la sintaxis correcta para restar los números de arr1 de los números de arr2?

a) `np.min(arr1, arr2)` b) `np.minus(arr1, arr2)` c) `np.sub(arr1, arr2)` d) `np.substract(arr1, arr2)`

In [45]:
```
# En NumPy la función se escribe np.subtract (la opción del quiz tiene una errata)
np.subtract(arr2, arr1)
```

Out[45]:
```
array([ 9, 18, 27])
```

Respuesta: d) `np.substract(arr1, arr2)`

17. ¿Cuál es el método correcto para redondear decimales en NumPy?

a) `np.fix()` b) `np.trunc()` c) Las otras 3 opciones son métodos válidos en NumPy d) `np.around()`

In [46]:
```
x = np.array([-3.1666, 3.6667])
print(np.fix(x), np.trunc(x), np.around(x, 2))
```

```
[-3.  3.] [-3.  3.] [-3.17  3.67]
C:\Users\Pc\AppData\Local\Temp\ipykernel_42044\2184682913.py:2: DeprecationWarning:
numpy.fix is deprecated. Use numpy.trunc instead, which is faster and follows the
Array API standard.
  print(np.fix(x), np.trunc(x), np.around(x, 2))
```

Respuesta: c) Las otras 3 opciones son métodos válidos en NumPy

18. ¿Cuál sería la respuesta de esta suma acumulativa en NumPy?

```
arr = np.array([1,2,3])
print(np.cumsum(arr))
```

a) `[3, 6, 9]` b) `[6]` c) `[9]` d) `[1 3 6]`

In [47]:
```
arr = np.array([1, 2, 3])
print(np.cumsum(arr))
```

```
[1 3 6]
```

Respuesta: d) `[1 3 6]`

19. ¿Cuál es la sintaxis correcta para crear una Series de pandas a partir de una lista?

a) `pd.Series(mylist)` b) `pd.createSeries(mylist)` c) `pd.getSeries(mylist)`

In [48]:
```
mylist = [1, 7, 2]
pd.Series(mylist)
```

Out[48]:
```
0    1
1    7
2    2
dtype: int64
```

Respuesta: a) `pd.Series(mylist)`